UNIFUCAMP

Ciência da Computação - 4º período

Programação Orientada a Objetos II

Threads em Java: pesquisa e simulador de corrida

Vinicius Marques Pedrosa Lino

Professor: Marcio Alves de Araujo

Trabalho II | Entrega indicada: 05/10/2026

1. Objetivo e escopo

Este relatório fundamenta um simulador de corrida no console em que cada carro executa uma tarefa independente. A aplicação relaciona encapsulamento, criação de threads, espera e proteção de dados compartilhados. O projeto inclui pódio sincronizado, pit stop e testes; a interface JavaFX foi omitida, conforme seu caráter opcional no enunciado [13].

A redação, a implementação e as verificações foram preparadas com auxílio substancial de IA. O material foi desenvolvido especificamente para este trabalho, sem reutilizar arquivos de outros chats. A conclusão é uma proposta para revisão do aluno. Como o layout oficial não foi fornecido, foi adotada formatação acadêmica simples; permanece pendente a adequação ao modelo institucional.

1.1. Processo, thread e concorrência

Um processo é uma instância de programa em execução, com recursos e espaço de memória próprios. Threads são fluxos de execução dentro de um processo: compartilham recursos, mas mantêm seu próprio caminho de execução. Esse compartilhamento facilita a comunicação e também exige cuidado quando várias threads alteram o mesmo objeto [1].

Concorrência permite que tarefas progridam em períodos sobrepostos; paralelismo físico corresponde à execução simultânea, possível quando há recursos de processamento disponíveis. Um único núcleo pode intercalar tarefas concorrentes. Portanto, linhas alternadas no console evidenciam intercalamento, mas não demonstram, por si só, processamento simultâneo em múltiplos núcleos [1].

No projeto, os carros pertencem ao mesmo processo Java. Cada objeto Carro mantém sua distância; todos compartilham um Podio. A classificação é o dado que precisa de coordenação entre participantes. Não são criados processos separados para cada veículo.

2. Formas de criar uma thread

Nas duas formas tradicionais solicitadas, a tarefa é definida em run(). Ao estender Thread, a classe sobrescreve esse método e pode chamar start() na própria instância. A vantagem é a simplicidade em

exemplos pequenos ou em uma especialização do mecanismo de thread. A desvantagem é ocupar a única herança de classe de Java e aproximar o modelo de domínio do controle de execução [2].

Ao implementar Runnable, a classe fornece run() e é passada ao construtor de Thread. Essa composição separa tarefa e execução, preserva a possibilidade de herdar outra classe e facilita testar o comportamento. Exige criar e coordenar um objeto Thread adicional. É a escolha adequada quando a classe representa uma tarefa de domínio e não uma especialização da infraestrutura [2].

Foi escolhido Runnable: Carro representa o participante, Corrida gerencia sua execução e Podio administra o resultado. Em Corrida.executar(), um laço instancia os carros, usa new Thread(carro, "piloto-" + i), guarda as referências em List<Thread> e chama start() em outro laço. Cada objeto Carro é usado por uma única thread e por uma única corrida.

3. Ciclo de vida

A API distingue seis estados da máquina virtual. Eles não são uma reprodução exata dos estados do sistema operacional [3].

Estado	Significado	Relação com o projeto
NEW	Criada, ainda não iniciada.	Entre new Thread(...) e start().
RUNNABLE	Executando ou apta a executar; pode aguardar CPU.	Sorteio e avanço de cada carro.
BLOCKED	Aguarda adquirir um monitor.	Possível disputa pelo synchronized(podio).
WAITING	Espera sem prazo por uma ação.	Espera da largada; join() sem tempo limite.
TIMED_WAITING	Espera com prazo.	sleep() do avanço e do pit stop.
TERMINATED	Execução finalizada.	run() retorna, inclusive após interrupção tratada.

Fluxo típico: NEW passa a RUNNABLE após start(); pode alternar com esperas e bloqueios; termina quando a execução acaba. Não é obrigatório visitar todos os estados. Uma thread terminada não pode ser reiniciada por outro start(); uma nova corrida exige novas instâncias de Thread [3, 4].

A associação dos estados ao código é conceitual. A suíte entregue não coleta todos os Thread.State em execução e não afirma ter observado cada um deles.

4. Métodos fundamentais e seus efeitos

Método	Efeito e aplicação

start()	Agenda a nova thread para executar run(). No projeto é chamado uma vez por participante [4].
run()	Contém a tarefa. Chamá-lo diretamente executa uma chamada comum na thread atual; não cria concorrência [4].
sleep(ms)	Pausa a thread atual por um intervalo sujeito ao temporizador e ao escalonador. Não é garantia de retorno no instante exato [5].
join()	Faz a thread chamadora aguardar o término da thread alvo. Corrida espera todos os carros antes de mostrar o resultado [6].

Substituir start() por run() em um exemplo comum faria os trabalhos ocorrerem na thread chamadora. Nesta implementação, a troca ainda impediria a coordenação: o primeiro run() aguardaria a largada e o coordenador não alcançaria o ponto de liberação. Por isso a escolha do método é uma condição funcional, não apenas uma diferença de estilo.

Cada iteração de Carro.run() sorteia um avanço, imprime o estado e chama Thread.sleep() com 100 a 500 ms. O pit acrescenta uma espera de 800 a 1200 ms. As pausas não consomem um laço de espera ativa. Caso sleep() ou await() lance InterruptedException, a implementação restaura a marca com Thread.currentThread().interrupt() e encerra a tarefa [4, 5, 10].

5. Condições de corrida

Uma condição de corrida ocorre quando o resultado depende de uma ordem não controlada de acessos concorrentes ao estado compartilhado. Uma instrução aparentemente simples pode envolver leitura, cálculo e escrita. Intercalar essas etapas permite perder uma atualização ou produzir um resultado inconsistente [7].

Exemplo ligado ao simulador: sem proteção, dois carros poderiam ler tamanho zero na lista, calcular ambos a posição 1 e depois inserir seus nomes. Proteger somente uma variável de posição não protegeria automaticamente a lista nem a associação entre nome e colocação. É necessário tratar a decisão inteira como uma operação coordenada.

Sequência ilustrativa: A lê quantidade 0; B também lê 0; A calcula posição 1; B calcula posição 1; ambos registram a mesma colocação. Trata-se de um exemplo conceitual de implementação incorreta, não de um erro observado na solução entregue. O monitor comum impede esse intercalamento do registro composto.

6. Sincronização e decisões de implementação

Um método de instância declarado synchronized adquire o monitor de this durante sua execução. Em um método static synchronized, o monitor pertence ao objeto Class. Um bloco synchronized(objeto) permite escolher o monitor e delimitar a região protegida. A exclusão mútua funciona entre acessos que usam o mesmo monitor [8, 9].

Além de evitar acessos simultâneos na região crítica, liberar um monitor e adquiri-lo posteriormente estabelece uma relação de visibilidade. A especificação também estabelece relações de ordem para start() e para a detecção do término por join(). Esperar um número arbitrário de milissegundos não substitui essas garantias [11].

6.1. Pódio e significado de cruzar a linha

Podio.registrar() usa synchronized(this) para verificar duplicação, inserir o nome e obter a colocação. classificacao() usa o mesmo monitor e retorna uma cópia imutável. Carro.run() usa synchronized(podio) ao atualizar a distância, imprimir o passo e registrar a chegada. Assim, o cruzamento na simulação e sua posição são definidos dentro da mesma região crítica.

A chamada de registrar() de dentro desse bloco volta a adquirir o mesmo monitor. Isso é permitido porque os monitores Java são reentrantes [9]. A proteção interna conserva a segurança de chamadas isoladas a Podio. O programa usa um monitor comum, sem encadear diferentes travas para o pódio.

sleep() não libera o monitor que uma thread já possui [11]. Por isso as pausas ficam fora do bloco. A escrita de status fica dentro para manter a sequência de avanço e chegada compreensível; isso serializa a saída e pode influenciar o escalonamento. É uma escolha didática para um console pequeno, não uma arquitetura de alto desempenho.

6.2. Largada, encapsulamento e aleatoriedade

Dois CountDownLatch coordenam o início: prontos começa com o número de carros e é decrementado na entrada de cada run(); largada começa em 1 e só é liberado depois que o coordenador recebeu todos os sinais. Nenhum avanço ocorre antes desse sinal comum. A liberação torna as tarefas elegíveis para prosseguir; não garante execução no mesmo instante físico [10].

Os campos nome, distanciaTotalCorrida e distanciaPercorrida são privados, sendo a distância inicial zero. O getter de distância adquire o monitor para consultas externas. O avanço sorteado varia de 20 a 80 metros, mas é limitado à metade antes do pit e ao total na chegada. Isso evita ultrapassar a pista e garante um pit único mesmo em percursos curtos.

Cada carro tem seu próprio Random, inicializado pela semente geral somada ao índice. Uma semente fixa preserva os sorteios locais; o sistema operacional continua decidindo a ordem das threads. AtomicReference apenas transporta eventual falha ao coordenador; não substitui a sincronização do pódio. Não foi necessário usar AtomicInteger ou ReentrantLock.

7. Verificação prática e evidências

As verificações abaixo foram executadas pelo assistente em 05/10/2026, em Windows com Oracle JDK 21.0.11. Fontes e testes compilaram com --release 17, -Xlint:all e -Werror, sem avisos na execução final. A opção release verifica o alvo e as APIs; não equivale a executar em uma JVM 17. Os registros integrais estão em evidencias/ no projeto.

Cenário	Passos	Trocas de carro	Pits	Tempo
Padrão: 4 carros, 1000 m	83	74	4	7,679 s

4 / 1000 / sim / 2026	87	76	4	8,695 s
3 / 180 / nao / 99	12	7	0	1,447 s
1 / 1 / sim / 42	2	0	1	1,941 s
8 / 125.5 / sim / 10	32	31	8	2,777 s
20 / 1 / nao / 7	20	19	0	0,689 s

Todos os seis cenários encerraram com código 0. A contagem de trocas compara carros de linhas PASSO consecutivas. O verificador conferiu o sinal de largada, as distâncias, os incrementos, a unicidade das chegadas, o top 3 e a igualdade entre chegada e classificação. Na corrida completa com semente 2026, a ordem foi Carro 3, Carro 2, Carro 4 e Carro 1.

7.1. Testes de concorrência e situações de borda

A suíte Java aprovou seis grupos: disputa do pódio em 10 rodadas de 64 threads; cópia imutável e rejeição de duplicação; retenção na largada e conclusão exata; interrupção antes da largada; interrupção durante o pit; validação de quantidades e distâncias. As verificações usam condições explícitas e geram AssertionError em caso de falha.

Nove chamadas adicionais da interface rejeitaram entradas inválidas com código 2: quantidades zero, excessiva e não numérica; NaN, distância negativa e infinito; opção de pit desconhecida; semente inválida; argumento excedente. A inspeção dos quatro arquivos Java confirmou ausência de comentários. resumo.json contém horários, durações, resultados e hashes SHA-256 dos fontes.

7.2. Limites da evidência

Os tempos incluem o início do processo e não são um benchmark. Não houve execução em Linux, macOS ou JVM 17; não foi feita medição exata das pausas nem observação de todos os estados. Testes finitos não cobrem todos os escalonamentos: a confiança no pódio depende também da análise do monitor comum. O intercalamento registrado demonstra concorrência observável, sem provar paralelismo físico.

Durante a preparação houve uma falha de escrita do compilador no ambiente isolado e um caminho de log incorreto. Ambos foram resolvidos; o histórico consta em TESTES.md. Nenhuma dessas tentativas foi contabilizada como teste funcional aprovado.

8. Conclusão

O desenvolvimento desta aplicação demonstrou na prática a importância de definir tarefas independentes e de estabelecer regras rigorosas para o acesso a recursos partilhados. No simulador, a separação entre o comportamento do carro, a coordenação da corrida e o registo do pódio foi fundamental. A escolha da interface `Runnable` tornou esta arquitetura mais modular e clara.

O principal desafio conceptual consistiu em distinguir **intercalamento**, paralelismo e atomicidade. Ficou evidente que uma saída alternada na consola não garante, por si só, que

uma atualização de dados esteja correta sem a devida sincronização. Além disso, a implementação exigiu uma distinção clara entre a pausa gerada por `sleep()` e a espera coordenada de `join()`, bem como a compreensão de que a invocação direta do método `run()` não inicia uma nova thread.

A estabilidade e a qualidade do simulador dependeram diretamente da correta delimitação das regiões críticas e da validação de cenários limite, provando que o comportamento concorrente exige um planeamento rigoroso para evitar resultados inconsistentes.

Referências

Fontes oficiais verificadas em 05/10/2026. Os Java Tutorials indicam origem no JDK 8; foram usados para conceitos fundamentais, em conjunto com a documentação da API e a especificação Java SE 21. As explicações e decisões do simulador foram redigidas para este projeto.

[1] ORACLE. Processes and Threads. The Java Tutorials. Disponível em: https://docs.oracle.com/javase/tutorial/essential/concurrency/procthread.html. Acesso em: 05 out. 2026.

[2] ORACLE. Defining and Starting a Thread. The Java Tutorials. Disponível em: https://docs.oracle.com/javase/tutorial/essential/concurrency/runthread.html. Acesso em: 05 out. 2026.

[3] ORACLE. Thread.State. Java SE 21 API. Disponível em: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Thread.State.html. Acesso em: 05 out. 2026.

[4] ORACLE. Thread. Java SE 21 API. Disponível em: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Thread.html. Acesso em: 05 out. 2026.

[5] ORACLE. Pausing Execution with Sleep. The Java Tutorials. Disponível em: https://docs.oracle.com/javase/tutorial/essential/concurrency/sleep.html. Acesso em: 05 out. 2026.

[6] ORACLE. Joins. The Java Tutorials. Disponível em: https://docs.oracle.com/javase/tutorial/essential/concurrency/join.html. Acesso em: 05 out. 2026.

[7] ORACLE. Thread Interference. The Java Tutorials. Disponível em: https://docs.oracle.com/javase/tutorial/essential/concurrency/interfere.html. Acesso em: 05 out. 2026.

[8] ORACLE. Synchronized Methods. The Java Tutorials. Disponível em: https://docs.oracle.com/javase/tutorial/essential/concurrency/syncmeth.html. Acesso em: 05 out. 2026.

[9] ORACLE. Intrinsic Locks and Synchronization. The Java Tutorials. Disponível em: https://docs.oracle.com/javase/tutorial/essential/concurrency/locksync.html. Acesso em: 05 out. 2026.

[10] ORACLE. CountDownLatch. Java SE 21 API. Disponível em: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/CountDownLatch.html. Acesso em: 05 out. 2026.

[11] ORACLE. The Java Language Specification, Java SE 21 Edition. Cap. 17: Threads and Locks, especialmente 17.1, 17.3 e 17.4.5. Disponível em: https://docs.oracle.com/javase/specs/jls/se21/html/jls-17.html. Acesso em: 05 out. 2026.

[12] GITHUB. Adding locally hosted code to GitHub. GitHub Docs. Fundamenta o guia de publicação que acompanha oprojeto. Disponível em: https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-impor t-source-code/adding-locally-hosted-code-to-github. Acesso em: 05 out. 2026.

[13] Trabalho II - Threads em Java: Pesquisa e Aplicação. Enunciado fornecido, 2 p. Arquivo:

Trabalho_II_Threads_Pesquisa_e_Aplicacao (1).pdf. Disciplina Programação Orientada a Objetos II, Ciência da Computação, 4º período. Entrega indicada: 05/10/2026.